

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

(Advanced Algorithm)

*** Eng. Firas Foad AL-Ejla ***

Answers of Assignment 1

Eng. Deema Mohammed AL-Maqadma

1. خوارزمية البحث الثنائي: (Binary Search)

• الفكرة/

- تبحث عن عنصر معين في مصفوفة مرتبة عن طريق تقسيمها لنصفين كل مرة، وتقارن العنصر المطلوب بالعنصر في المنتصف. إذا لم تجده، تكرر العملية في النصف المناسب حتى تجد العنصر أو تنتهي العناصر.
- (باختصار نرتب المصفوفة ونبحث عن العنصر في المنتصف، إذا مو هو المطلوب نكرر على النصف المناسب، وهكذا حتى نلاقيه أو تنتهي العناصر)

- خطوات التطبيق الخوارزمية على المصفوفة :

array : [15, 7, 22, 3, 9, 5]

- 1. لازم نرتب المصفوفة أولاً : [3, 5, 7, 9, 15, 22]
- 2. لنبحث عن 9 مثلاً، نأخذ المنتصف (العنصر 9)، وجدناه (اي تساوت قيمة العنصر الأوسط mid مع قيمة العنصر الذي نبحث target بالتالي ينتهي البحث)، في حال لم نجده، نقسم المصفوفة لقسمين بناء على العنصر الأوسط mid ونقارن قيمة العنصر الذي نبحث عنه target بقيمة mid لمعرفة المكان المناسب للبحث ، في حال كان $mid > target$ نبحث في الجزء الأصغر (اي من left الى $mid-1$) أما في حال العكس $mid < target$ نبحث في الجزء الأكبر (اي من $mid+1$ إلى right)، ونكرر العملية حتى نجد العنصر المطلوب في مكانه

- التعقيد الزمني / Time Complexity

• $O(\log n)$

- لأننا نقسم المصفوفة لنصفين في كل خطوة أي يقل عدد العناصر للنصف $n/2$

كود بلغة الجافا يوضح عمليا تطبيق خوارزمية (Binary Search)

```
// Deema Mohammed AL-Maqadma
public class BinarySearch {
    public static void binarySearch(int[] arr, int target) {
        int left = 0; // ترتيب العنصر الأول
        int right = arr.length - 1; // ترتيب العنصر الأخير
        boolean isFaound = false;

        while (left <= right) {
            int mid = left + (right - left) / 2; // ترتيب العنصر الاوسط

            if (arr[mid] == target) {
                isFaound = true;
                break;
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        if (isFaound) {
            System.out.println("Target : " + target + "is Faound");
        } else {
            System.out.println("Target : " + target + "is NOT Faound");
        }
    }
}
```

2. خوارزمية فرز الفقاعة: (Bubble Sort)

• الفكرة/

- تمر على المصفوفة عدة مرات، وتبدل swap كل عنصر مع العنصر الذي يليه إذا كان أكبر منه، حتى تصبح المصفوفة مرتبة تصاعدياً ، وفي هذه الخوارزمية بركز على وضع العنصر الأكبر في مكانه الصحيح اي نبدأ بترتيب العناصر بحيث نضع أولاً العنصر الأكبر Largest في مكانه الصحيح ثم نكمل حتى تترتب المصفوفة (باختصار نمر على المصفوفة أكثر من مرة، كل مرة نبذل بين كل عنصر وجاره إذا كان أكبر، ونكرر حتى تصير مرتبة بالكامل.)

Note /

- مقارنة بين خوارزميات البحث / Searching
- خوارزمية الفرز بالاختيار (Selection Sort) وخوارزمية فرز الفقاعة (Bubble Sort) :
- تعتبر خوارزمية Bubble Sort أذكى ، لأنه في حال كانت المصفوفة مرتبة لن يحدث اي تبديل swap بالتالي ما يحتاج أمر على عناصر المصفوفة مرة اخرى

- خطوات التطبيق الخوارزمية على المصفوفة :

- array : [15, 7, 22, 3, 9, 5]
- 1. قارن ١٥ و ٧ → بدّل
- 2. قارن ١٥ و ٢٢ → لا تبديل
- 3. قارن ٢٢ و ٣ → بدّل
- 4. كمل بنفس الطريقة حتى نهاية المصفوفة وكرر عدة مرات حتى الترتيب النهائي [3, 5, 7, 9, 15, 22]

• - التعقيد الزمني / Time Complexity

- $O(n^2)$ لأنك تمر على كل العناصر عدة مرات

كود بلغة الجافا يوضح عمليا تطبيق خوارزمية (Bubble Sort)

```
// Deema Mohammed AL-Maqadma
public class BubbleSort {

    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

» وَاٰخِرُ دَعْوَاهُمْ اَنْ الْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِينَ »